Inferencia local de LLM: Restricciones arquitectónicas y estrategias de optimización para la informática personal

Autor: Oscar Gallardo
Afiliación: Proyecto lap-llm / Hackatón CubePath 2026
Fecha: Marzo 2026

Resumen

El acceso a modelos de lenguaje de gran escala (LLM) ha quedado tradicionalmente restringido a infraestructuras cloud o estaciones de trabajo de alto rendimiento. Este trabajo presenta un marco práctico para democratizar el acceso a la IA local mediante la ejecución de LLMs en hardware modesto — específicamente laptops con GPUs integradas (iGPUs) bajo sistemas Linux.

Proponemos una **arquitectura de tres capas** que integra:

1. **Optimización del kernel Linux y gestión de memoria unificada (UMA).**
2. **Backends de inferencia heterogéneos** (SYCL para Intel, ROCm/HIP para AMD).
3. **Estrategias de cuantización adaptativas.**

Demostramos que mediante la comprensión profunda de los mecanismos de memoria del kernel — incluyendo PPGTT, TTM, y políticas NUMA — es posible ejecutar modelos de **mas capaces en laptops con 8-32GB RAM** sin degradación crítica del rendimiento.

Nuestro enfoque prioriza la **soberanía digital**: el usuario mantiene control total sobre sus datos y dependencias tecnológicas, eliminando la necesidad de conectividad cloud o suscripciones a APIs propietarias. Este trabajo establece las bases técnicas para sistemas de asesoramiento automatizado de hardware, validando que la IA útil es accesible más allá de las fronteras del hardware premium.

---

1. Introducción

- **Contexto:** La centralización de la IA en grandes proveedores cloud crea dependencias tecnológicas y vulnerabilidades de privacidad. Paralelamente, la narrativa técnica asume erróneamente que la inferencia local de LLMs requiere GPUs dedicadas de alto consumo.
- **Problema:** Falta de documentación sistemática sobre cómo aprovechar arquitecturas de memoria unificada y backends de cómputo heterogéneo en hardware común.

- **Contribución:** Un marco técnico verificado para ejecutar LLMs capaces en laptops con iGPUs, con énfasis en optimizaciones a nivel de kernel y gestión de memoria.

I. Fundamentos de Gestión de Memoria en Linux

1. La UMA como Recurso Elástico para Inferencia

Las iGPUs no tienen VRAM dedicada. Utilizan **Arquitectura de Memoria Unificada (UMA)**: comparten la RAM del sistema con la CPU, y el kernel asigna dinámicamente una porción como memoria de video.

Para LLMs, esto es crítico: Pues por defecto en Laptops se suele asignar entre 500MB y 2G de RAM física a la iGPU, lo cual es insuficiente para modelos más capaces.

2. Mecanismos de Memoria GPU Relevantes para LLMs

No toda la gestión de memoria Linux es igual de relevante para inferencia. Nos enfocamos principalmente en el uso de **Páginas:** La unidad básica de gestión es el marco de página, que suele ser un bloque fijo de 4 KB, de tal manera que podamos reproducir un pool de memoria.

TTM (Translation Table Maps) - AMD El subsistema de memoria gráfica de AMD que gestiona la memoria de GPU como un *memory pool* compartido. * **Función para LLMs:** Permite que el driver `amdgpu` asigne más memoria de la que la BIOS reserva como "VRAM estática". * **Mecanismo clave:** `amdgpu.vm_update_mode` y políticas de asignación dinámica permiten que la iGPU use RAM del sistema bajo demanda. * **Resultado práctico:** En el sistema de prueba, esto permitió cargar modelos de 10G a 18G, gracias al uso de un sistema minimalista en Gentoo.

Para las pruebas de este estudio, yo use una thinkpad con amd.

II. Implementación y Optimización en iGPUs AMD

1. Entorno de Validación y Hardware

Para validar las estrategias de optimización, se utilizó una plataforma de hardware de consumo con las siguientes especificaciones:

- **CPU**: AMD Ryzen 7 PRO 4750U (8C/16T, Zen 2)
- **iGPU**: AMD Radeon Vega 8 (Arquitectura nativa gfx90c)
- **Memoria RAM**: 32GB DDR4 (~29GB usables para el sistema)
- **Sistema Operativo**: Gentoo Linux con kernel 6.x personalizado

Nota sobre compatibilidad: Debido a que la arquitectura Renoir (gfx90c) carece de soporte oficial en los binarios de ROCm, se realizó una compilación orientada a gfx900 (Vega 64) utilizando el flag `AMDGPU_TARGETS=gfx900`. La ejecución se garantiza mediante la variable de entorno `HSA_OVERRIDE_GFX_VERSION=9.0.0`, permitiendo el despacho de kernels HIP en hardware no listado.

2. Arquitectura del Modelo: Qwen3-30B-A3B

El despliegue de un modelo de 30 mil millones de parámetros en una laptop es posible gracias a la arquitectura Mixture-of-Experts (MoE) y técnicas de cuantización avanzada.

Especificación	Valor	Implicación Técnica
Parámetros Totales	30B	Alta capacidad de razonamiento y conocimiento
Parámetros Activos	~3B	Carga computacional reducida por token (baja latencia)
Cuantización	IQ4_XS	Reducción de precisión a ~4 bits con mínima pérdida
Peso en Disco (W_{model})	16.4 GB	Tamaño real del archivo GGUF mapeado en memoria

3. Marco Matemático para la Gestión de Memoria

Para garantizar la estabilidad del sistema y evitar el OOM (Out of Memory) Killer, la asignación de memoria se divide en tres etapas de cálculo:

A. Cálculo de Disponibilidad Dinámica (Steady State) Determina cuánta memoria física puede ser cedida a la GPU sin colapsar el sistema operativo.

$$VRAM_{usable} = RAM_{libre} - Margen_{seguridad}$$

Donde el Margen de seguridad se establece en **1.5 GB** para operaciones críticas del Kernel.

B. Requerimiento Total de Inferencia ($VRAM_{req}$) Suma el peso estático del modelo y la memoria volátil del contexto.

$$VRAM_{req} = W_{model} + KV_{cache} + \delta$$

- W_{model}: 16.4 GB
- δ: Overhead de buffers (aprox. 1.0 GB)
- KV_{cache}: Se calcula según la arquitectura del modelo:

$$KV_{cache} \approx \frac{2 \times L \times H \times D \times C \times P}{1024^3}$$

(Para Qwen3-30B con 8k tokens y precisión FP16, este valor es de **1.5 GB**)

C. Conversión a Páginas del Kernel (TTM Pages) El parámetro `amdttm.pages_limit` requiere el valor en unidades de páginas (4 KB en x86_64).

$$TTM_{pages} = \frac{VRAM_{req} \times 1024 \times 1024}{4}$$

Ejemplo de validación para 30B (8k contexto):

$$VRAM_{req} = 16.4 + 1.5 + 1.0 = \mathbf{18.9} \text{ GB}$$

$$TTM_{pages} = \frac{18.9 \times 1,048,576}{4} = \mathbf{4,954,522} \text{ páginas}$$

4. Resultados: Comparativa de Capacidades

La transición de una gestión basada en BIOS a una gestión basada en el subsistema TTM del Kernel permite un salto generacional en la capacidad de inferencia local.

Parámetro	Configuración Estándar (BIOS)	Configuración Optimizada (TTM)
Método de Reserva	UMA Framebuffer (Estático)	amdttm.pages_limit (Dinámico)
Límite de VRAM	2 GB fijos	20 GB+ (según disponibilidad)
Modelo Máximo	~1.5B (Qwen2.5 1.5B)	30B (Qwen3-30B-A3B)
Contexto Viable	2,048 tokens	8,192+ tokens
Impacto en RAM	2GB bloqueados permanentemente	0GB en reposo; se asigna al cargar el modelo

III. Estrategias para iGPUs Intel: Estado del Arte e Hipótesis de Investigación

Nota metodológica: Esta sección se basa en análisis de documentación técnica, reportes de la comunidad y arquitectura de drivers, dado que el trabajo experimental se realizó en plataforma AMD. Se propone una hipótesis operativa para investigación futura.

1. Backend de Inferencia: SYCL sobre OpenCL

Para iGPUs Intel, la opción técnicamente madura es llama.cpp con backend SYCL, abandonando OpenCL por limitaciones de rendimiento.

Aspecto	Especificación
Requisito mínimo	Intel Core 11ª generación (arquitectura Xe)
Stack de software	Intel oneAPI Base Toolkit (compilador SYCL + oneMKL)
Ventaja clave	Vectorización eficiente de operaciones de atención
Limitación observada	Soporte discontinuado para arquitecturas Gen9 y anteriores

Comparativa con AMD: Mientras AMD requiere compilación de ROCm con targets específicos (gfx900), Intel proporciona oneAPI como distribución binaria, reduciendo la fricción inicial pero limitando la optimización a bajo nivel.

2. Gestión de Memoria: PPGTT vs. TTM

La divergencia arquitectónica entre Intel y AMD se manifiesta críticamente en el subsistema de memoria GPU:

Característica	AMD (TTM)	Intel (PPGTT + GEM)
Subsistema	Translation Table Manager	Per-Process Graphics Translation Table + Graphics Execution Manager
Control de pool	`amdttm.pages_limit` (parámetro kernel)	No existe equivalente directo documentado
Naturaleza	Pool paginado pre-reservable	Mapeo virtual por contexto
Límite práctico	Configurable hasta ~80% RAM	~50% RAM física (reportado)
Granularidad	Páginas de 4KB asignables bajo demanda	Buffer Objects (BO) gestionados por GEM

El problema del 50% en Intel

Documentación de la comunidad y manuales de referencia indican que iGPUs Intel en Linux típicamente operan con un techo del 50% de RAM física asignable como memoria gráfica. En un sistema con 32GB RAM, esto implica ~16GB

máximo — suficiente para modelos 7B-13B, pero restrictivo comparado con los 20GB logrados vía TTM en AMD.

Estrategia documentada para mitigar: Aumento físico de RAM a 32GB, permitiendo que el 50% asignable (~16GB) sea funcionalmente equivalente al límite TTM configurado en sistemas AMD con menos RAM total.

3. Hipótesis: Hacia un "PPGTT Ballooning"

Basado en el análisis de arquitectura de drivers y técnicas de virtualización (Intel GVT-g), se plantea la siguiente hipótesis de investigación:

Hipótesis: Es posible extender el límite efectivo de memoria GPU en iGPUs Intel mediante la manipulación de parámetros del driver i915 relacionados con el espacio de direcciones PPGTT, o mediante técnicas de "address space ballooning" adaptadas de entornos virtualizados.

Fundamento técnico: PPGTT proporciona aislamiento de direcciones virtuales por contexto, no reserva física. Sin embargo, el driver GEM subyacente asigna páginas físicas de memoria del sistema (shmem) a Buffer Objects bajo demanda. La barrera del 50% no es hardware en la mayoría de implementaciones Linux, sino una heurística del driver para evitar contención con el sistema operativo.

Lineas de investigación propuestas:

Enfoque	Mecanismo	Estado
Boot parameter	`i915.enable_ppgtt=2` (Full PPGTT) + modificadores de límite	No documentado para expansión de memoria
Debugfs	Escritura en `/sys/kernel/debug/dri/*/i915_gem_objects`	Requiere privilegios, formato no estandarizado
Cgroups v2	Limitación inversa: restringir memoria del sistema para forzar asignación GPU	Hipótesis no validada
Virtualización	Técnicas GVT-g de "ballooning" adaptadas a bare-metal	Complejidad alta, portabilidad dudosa

Diferencia fundamental con AMD:

```
AMD TTM:   Gestiona POOL de páginas físicas pre-asignables
           → amdttm.pages_limit controla el tamaño del pool

Intel GEM: Gestiona BO (Buffer Objects) mapeados vía PPGTT
           → El "pool" es la memoria del sistema menos usada por CPU
           → No existe parámetro directo para forzar reserva mínima/máxima
```

4. Configuración Recomendada (Basada en Documentación)

Para usuarios con iGPUs Intel que busquen replicar los resultados AMD sin modificaciones de kernel:

Componente	Recomendación	Rationale
RAM física	32GB mínimo	Eleva el 50% asignable a 16GB efectivos
Swap	32GB en SSD NVMe	Permite mmap de modelos grandes sin OOM
Cuantización	INT4 obligatorio (IQ4_XS/Q4_0)	Reduce 7B dense a ~4GB, 7B MoE a ~3GB activos
Contexto	4096-6144 tokens	Balance entre capacidad y presión de KV cache
Herramienta	Ollama + llama.cpp (SYCL) o BigDL-LLM	Abstracción de complejidad oneAPI

5. Trabajo Futuro: Validación de la Hipótesis PPGTT

Se propone como línea de investigación inmediata:

1. **Investigación de parámetros i915:** Revisión exhaustiva de parámetros de módulo no documentados en `modinfo i915`
2. **Instrumentación del kernel:** Uso de ftrace y perf para mapear rutas de asignación GEM bajo carga de llama.cpp
3. **Prueba de concepto:** Modificación de `drivers/gpu/drm/i915/gem/i915_gem_shmem.c` para eliminar o parametrizar la heurística del 50%
4. **Comparativa SYCL vs. ROCm:** Benchmark de throughput (tokens/segundo) en hardware comparable (Core i7-1165G7 vs. Ryzen 7 PRO 4750U)

6. Conclusión de Sección

Mientras que en AMD el subsistema TTM permite configuración explícita del pool de memoria GPU mediante parámetros de kernel, en Intel la gestión es implícita a través de GEM y limitada por heurísticas del driver. La hipótesis de un "PPGTT ballooning" permanece sin validar, siendo la estrategia operativa actual la maximización de RAM física para elevar el límite del 50% a valores funcionalmente equivalentes.

Bibliografía

1. Drepper, U. (2007, 21 de septiembre). *What every programmer should know about memory.* Red Hat.

2. Gale, T. (2025, 3 de abril). *Efficient Mixture of Experts models*. Google DeepMind.
3. Grootendorst, M. (s.f.). *A Visual Guide to Mixture of Experts (MoE)*. Recuperado de https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mixture-of-experts
4. *The Translation Table Manager (TTM)*. (s.f.). Recuperado de https://www.kernel.org/doc/html/latest/gpu/drm-mm.html
5. *Run LLMs on Intel® GPUs Using llama.cpp*. (s.f.). Intel. Recuperado de https://www.intel.com/content/www/us/en/developer/articles/technical/run-llms-on-gpus-using-llama-cpp.html
6. *drm/i915 Intel GFX Driver*. (s.f.). Recuperado de https://www.kernel.org/doc/html/latest/gpu/i915.html
7. Geerling, J. (2025). *Increasing VRAM allocation on AMD AI APUs under Linux*. Recuperado de https://www.jeffgeerling.com/blog/2025/increasing-vram-allocation-on-amd-ai-apus-under-linux/
8. Funnell, J. (s.f.). *Industrial LLM inference on a student's laptop?* Medium. Recuperado de https://medium.com/@john.funnell/industrial-llm-inference-on-a-students-laptop-a090a1bd1b2c